PKG 17: GOOGLE GUAVA Search | GOOGLE File I/O | GOOGLE Graphs

```
/**
 * Google Guava Search
 *
 * Google Guava 29.0 2020
 *
*/

import com.google.common.collect.ArrayListMultimap;
import com.google.common.collect.Multimap;

public class DB220 { // Dictionary220

    private enum Entry {

        BOOK("book", "A set of pages.");

        private String key;
        private String value;

        private Entry() {
        }

        private Entry(String key, String value) {
            this.key = key;
            this.value = value;
        }

        public String getKey() {
            return this.key;
        }

        public String getValue() {
            return this.value;
        }
    }

    public static void main(String[] args) {

        Multimap<String, String> dictionary = ArrayListMultimap.create();

        // Adding some key/value
        dictionary.put(DB220.Entry.BOOK.getKey(), DB220.Entry.BOOK.getValue());

        // Search
        String searchKey = "book";
        if (dictionary.containsKey(searchKey)) {
            System.out.println(searchKey + "\t <Found>");
        } else {
            System.out.println(searchKey + "\t <Not found>");
        }
    }
}

// book      <Found>
```

```
/**
 * Google Guava IO
 *
 * Google Guava 29.0 2020
*/

import com.google.common.io.Files;
import com.google.common.io.MoreFiles;

import java.io.File;
import java.io.IOException;
import java.nio.file.Path;
import java.nio.file.Paths;

public class DirListingGG29 {

    public static void main(String args[]) throws IOException {

        String rootDir = "./src/pkg17/Level-1-Root";
        Path path = Paths.get(rootDir);
        File file = new File(rootDir);

        System.out.println("getChildren ====================================");
#1      getChildren(path);

        System.out.println("breadthFirst ===================================");
#2      breadthFirst(file);

        System.out.println("depthFirstPreOrder ==============================");
#3      depthFirstPreOrder(file);

        System.out.println("depthFirstPostOrder ==============================");
#4      depthFirstPostOrder(file);
    }

    public static void getChildren(Path rootDir) throws IOException {
        for (Path file : MoreFiles.listFiles(rootDir)) {
            System.out.println(file);
        }
    }

    public static void breadthFirst(File rootDir) {
        for (File file : Files.fileTraverser().breadthFirst(rootDir)) {
            System.out.println(file);
        }
    }

    public static void depthFirstPreOrder(File rootDir) {
        for (File file : Files.fileTraverser().depthFirstPreOrder(rootDir)) {
            System.out.println(file);
        }
    }

    public static void depthFirstPostOrder(File rootDir) {
        for (File file : Files.fileTraverser().depthFirstPostOrder(rootDir)) {
            System.out.println(file);
        }
    }
}
```

```
// The current working directory = ./src/pkg
// Directory "Level-1-Root" is in pkg
```

```
├───Level-1-Root
│   ├───Level-2-A
│   │   ├───Level-3-A-1
│   │   │       Mickey.txt
│   │   │
│   │   └───Level-3-A-2
│   │           Mickey.txt
│   │
│   └───Level-2-B
│       ├───Level-3-B-1
│       │       Mickey.txt
│       │
│       └───Level-3-B-2
│               Mickey.txt
│
└───Level-1-Root-BigTree
```

```
getChildren ====================================
.\src\pkg17\Level-1-Root\Level-2-A
.\src\pkg17\Level-1-Root\Level-2-B
```

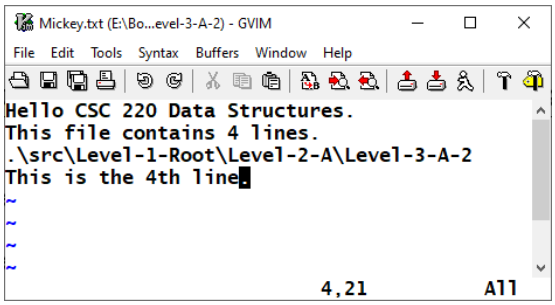
```
breadthFirst ===================================
.\src\pkg17\Level-1-Root
.\src\pkg17\Level-1-Root\Level-2-A
.\src\pkg17\Level-1-Root\Level-2-B
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-1
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-2
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-1
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-2
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-1\Mickey.txt
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-2\Mickey.txt
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-1\Mickey.txt
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-2\Mickey.txt
```

```
└──Level-1-Root-BigTree
   ├──Level-2-A
   │  ├──Level-3-A-1
   │  │  └──Level-4-A
   │  │     └──Level-5-A
   │  │        └──Level-6-A
   │  │           └──Level-7-A
   │  │              └──Level-8-A
   │  │                 └──Level-9-A
   │  │                       Mickey.txt
   │  │
   │  └──Level-3-A-2
   │        Mickey.txt
   │
   ├──Level-2-B
   │  ├──Level-3-B-1
   │  │     Mickey.txt
   │  │
   │  └──Level-3-B-2
   │        Mickey.txt
   │
   ├──Level-2-C
   │  ├──Level-3-C-1
   │  │     Mickey.txt
   │  │
   │  └──Level-3-C-2
   │        Mickey.txt
   │
   ├──Level-2-D
   │  ├──Level-3-D-1
   │  │     Mickey.txt
   │  │
   │  └──Level-3-D-2
   │        Mickey.txt
   │
   └──Level-2-E
      ├──Level-3-E-1
      │     Mickey.txt
      │
      └──Level-3-E-2
            Mickey.txt
```

```
depthFirstPreOrder ==============================
.\src\pkg17\Level-1-Root
.\src\pkg17\Level-1-Root\Level-2-A
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-1
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-1\Mickey.txt
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-2
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-2\Mickey.txt
.\src\pkg17\Level-1-Root\Level-2-B
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-1
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-1\Mickey.txt
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-2
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-2\Mickey.txt
```

```
depthFirstPostOrder ==============================
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-1\Mickey.txt
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-1
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-2\Mickey.txt
.\src\pkg17\Level-1-Root\Level-2-A\Level-3-A-2
.\src\pkg17\Level-1-Root\Level-2-A
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-1\Mickey.txt
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-1
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-2\Mickey.txt
.\src\pkg17\Level-1-Root\Level-2-B\Level-3-B-2
.\src\pkg17\Level-1-Root\Level-2-B
.\src\pkg17\Level-1-Root
```

```java
/**
 * Google Guava IO
 *
 * Google Guava 29.0 2020
*/

import com.google.common.io.Files;

import java.io.File;
import java.io.IOException;
import java.nio.charset.Charset;

public class FileGG29 {

    public static void main(String args[]) throws IOException {

        String absolutePath = "X://AWS/CSC220/src/pkg17/Level-1-Root/Level-2-A/Level-3-A-2/Mickey.txt";
        String relativePath = "./src/pkg17/Level-1-Root/Level-2-A/Level-3-A-2/Mickey.txt";
        File mickeyTxt = new File(relativePath);

        System.out.println("File Name: " + Files.getNameWithoutExtension(relativePath));
        System.out.println("File Extension: " + Files.getFileExtension(relativePath));
        // File Name: Mickey
        // File Extension: txt

        System.out.println("Simplified Path: " + Files.simplifyPath(relativePath));
        System.out.println("Files equal: " + Files.equal(mickeyTxt, mickeyTxt)); // same bytes
        // Simplified Path: src/pkg17/Level-1-Root/Level-2-A/Level-3-A-2/Mickey.txt
        // Files equal: true

        for (String line : Files.readLines(mickeyTxt, Charset.defaultCharset())) {
            System.out.println(line);
        }
        // Hello CSC 220 Data Structures.
        // This file contains 4 lines.
        // .\src\Level-1-Root\Level-2-A\Level-3-A-2
        // This is the 4th line.
    }
}
```



```
/**
 * GOOGLE Graphs
 *

                   DineyStar


                  1 Mickey
                      /\
          3 Goofy --- /--\--- 4 Dory
                     /    \
                    /      \
                5 Nemo     2 Minnie


            Goofy and Dory are connected.
            Minnie and Nemo are connected.

 */

import com.google.common.graph.MutableValueGraph;
import com.google.common.graph.ValueGraphBuilder;
import java.util.Objects;

public class DisneyStar {

    private final int starNum;
    private final String starName;

    DisneyStar(int starNum, String starName) {
        this.starName = Objects.requireNonNull(starName, "Canot be null");
        this.starNum = starNum;
    }

    public int getVerticeNum() {
        return starNum;
    }

    public String getVerticeName() {
        return starName;
    }

    @Override
    public boolean equals(Object star) {
        if (star instanceof DisneyStar) {
            DisneyStar anotherStar = (DisneyStar) star;
            return this.starName.equals(anotherStar.starName)
                    && this.starNum == anotherStar.starNum;
        }
        return false;
    }

    @Override
    public int hashCode() {
        return Objects.hash(this.starName, this.starNum);
    }

    @Override
    public String toString() {
        return "(" + this.starNum + ", " + this.starName + ")";
    }
```

```
    public static void main(String args[]) {
        MutableValueGraph<DisneyStar, Integer> disneyStar = ValueGraphBuilder.directed().build();

        DisneyStar mickey = new DisneyStar(1, "Mickey");
        DisneyStar minnie = new DisneyStar(2, "Minnie");
        DisneyStar goofy = new DisneyStar(3, "Goofy");
        DisneyStar dory = new DisneyStar(4, "Dory");
        DisneyStar nemo = new DisneyStar(5, "Nemo");

        System.out.println("\n addNode(N Node)================================");
        disneyStar.addNode(mickey);
        disneyStar.addNode(minnie);
        disneyStar.addNode(goofy);
        disneyStar.addNode(dory);
        disneyStar.addNode(nemo);

        System.out.println("\n putEdgeValue(N nodeU, N nodeV, V value)=======");
        disneyStar.putEdgeValue(mickey, minnie, 10);
        disneyStar.putEdgeValue(minnie, goofy, 10);
        disneyStar.putEdgeValue(goofy, dory, 10);
        disneyStar.putEdgeValue(dory, nemo, 10);
        disneyStar.putEdgeValue(nemo, mickey, 10);

        System.out.println("\n edges()=========================================");
        // Returns all edges in this graph.
        disneyStar.edges().forEach((edge) -> {
            System.out.println(edge);
        });
        // <(1, Mickey) -> (2, Minnie)>
        // <(2, Minnie) -> (3, Goofy)>
        // <(3, Goofy) -> (4, Dory)>
        // <(4, Dory) -> (5, Nemo)>
        // <(5, Nemo) -> (1, Mickey)>

        System.out.println("\n degree(N node)================================");
        // Returns the count of node's incident edges
        System.out.println(disneyStar.degree(mickey)); // 2
        System.out.println(disneyStar.degree(minnie)); // 2
        System.out.println(disneyStar.degree(goofy));  // 2
        System.out.println(disneyStar.degree(dory));   // 2
        System.out.println(disneyStar.degree(nemo));   // 2

        System.out.println("\n nodes()========================================");
        // Returns all nodes in this graph.
        disneyStar.nodes().forEach((star) -> {
            System.out.println(star);
        });
        // (1, Mickey) // (2, Minnie) // (3, Goofy) // (4, Dory) // (5, Nemo)

        System.out.println("\n adjacentNodes(N node)=========================");
        // has an incident edge in common with
        disneyStar.adjacentNodes(mickey).forEach((star) -> {
            System.out.println(mickey + " w/ " + star + "\t");
        });
        // (1, Mickey) w/ (2, Minnie)
        // (1, Mickey) w/ (5, Nemo)

        disneyStar.adjacentNodes(minnie).forEach((star) -> {
            System.out.println(minnie + " w/ " + star + "\t");
        });
        // (2, Minnie) w/ (3, Goofy  // (2, Minnie) w/ (1, Mickey)
```

```
        System.out.println("\n edgeValueOrDefault(...)=======================");
        System.out.println("\n hasEdgeConnecting(N nodeU, N nodeV)===========");
        // Returns the value of the edge connecting nodeU to nodeV,
        // if one is present; otherwise, returns defaultValue.
        System.out.println(disneyStar.edgeValueOrDefault(mickey, goofy, 0));  //  0
        System.out.println(disneyStar.edgeValueOrDefault(mickey, minnie, 0)); // 10
        System.out.println(disneyStar.hasEdgeConnecting(mickey, goofy));      // false

        System.out.println("\n inDegree(N node)==============================");
        // Returns the count of node's incoming edges
        System.out.println(disneyStar.isDirected());      // true
        System.out.println(disneyStar.inDegree(mickey));  // 1
        System.out.println(disneyStar.outDegree(mickey)); // 1
        disneyStar.predecessors(mickey).forEach((star) -> {
            System.out.println(star + " --> " + mickey);
        });
        // (5, Nemo) --> (1, Mickey)
        disneyStar.successors(mickey).forEach((star) -> {
            System.out.println(mickey + " --> " + star);
        });
        // (1, Mickey) --> (2, Minnie)
        disneyStar.successors(minnie).forEach((star) -> {
            System.out.println(minnie + " --> " + star);
        });
        // (2, Minnie) --> (3, Goofy)
        disneyStar.successors(goofy).forEach((star) -> {
            System.out.println(goofy + " --> " + star);
        });
        // (3, Goofy) --> (4, Dory)
        disneyStar.successors(dory).forEach((star) -> {
            System.out.println(dory + " --> " + star);
        });
        // (4, Dory) --> (5, Nemo)

        System.out.println("\n asGraph=======================================");
        System.out.println(disneyStar.asGraph());
        // isDirected: true, allowsSelfLoops: false,
        // nodes: [(1, Mickey), (2, Minnie), (3, Goofy), (4, Dory), (5, Nemo)],
        // edges: [<(1, Mickey) -> (2, Minnie)>, <(2, Minnie) -> (3, Goofy)>,
        // <(3, Goofy) -> (4, Dory)>, <(4, Dory) -> (5, Nemo)>,
        // <(5, Nemo) -> (1, Mickey)>]

        System.out.println("\n Traverse directed graph edge-wise=============");
        danceWithTheStars(disneyStar, mickey);
        // (1, Mickey) --> (2, Minnie)
        // (2, Minnie) --> (3, Goofy)
        // (3, Goofy) --> (4, Dory)
        // (4, Dory) --> (5, Nemo)
        // (5, Nemo) --> (1, Mickey)
    }

    public static void danceWithTheStars(MutableValueGraph<DisneyStar, Integer>
                                                        disneyStar, DisneyStar star) {
        disneyStar.successors(star).forEach((nextStar) -> {
            if (star.starNum < nextStar.starNum) {
                System.out.println(star + " --> " + nextStar);
                danceWithTheStars(disneyStar, nextStar);
            } else {
                System.out.println(star + " --> " + nextStar);
            }
        });
    }
}
```

com.google.common.annotations	Common annotation types.
com.google.common.base	Basic utility libraries and interfaces.
com.google.common.cache	This package contains caching utilities.
com.google.common.collect	This package contains generic collection interfaces and implementations, and other utilities for working with collections.
com.google.common.escape	Interfaces, utilities, and simple implementations of escapers and encoders.
com.google.common.eventbus	The EventBus allows publish-subscribe-style communication between components without requiring the components to explicitly register with one another (and thus be aware of each other).
com.google.common.graph	An API for representing graph (node and edge) data.
com.google.common.hash	Hash functions and related structures.
com.google.common.html	Escapers for HTML.
com.google.common.io	This package contains utility methods and classes for working with Java I/O; for example input streams, output streams, readers, writers, and files.
com.google.common.math	Arithmetic functions operating on primitive values and `BigInteger` instances.
com.google.common.net	This package contains utility methods and classes for working with net addresses (numeric IP and domain names).
com.google.common.primitives	Static utilities for working with the eight primitive types and `void`, and value types for treating them as unsigned.
com.google.common.reflect	This package contains utilities to work with Java reflection.
com.google.common.util.concurrent	Concurrency utilities.
com.google.common.xml	Escapers for XML.

Guava is a set of core Java libraries from Google that includes new collection types (such as multimap and multiset), immutable collections, a graph library, and utilities for concurrency, I/O, hashing, caching, primitives, strings, and more! It is widely used on most Java projects within Google, and widely used by many other companies as well.

Google Guava can be roughly divided into three components: basic utilities to reduce menial labor to implement common methods and behaviors, an extension to the Java collections framework (JCF) formerly called the Google Collections Library, and other utilities which provide convenient and productive features such as functional programming, graphs, caching, range objects, and hashing.

The creation and architecture of the collection component were partly motivated by generics introduced in JDK 1.5. Although generics improve the productivity of programmers, the standard JCF does not provide sufficient functionality, and its complement Apache Commons Collections had not adopted generics in order to maintain backward compatibility. This fact led two engineers Kevin Bourrillion and Jared Levy to develop an extension to JCF, which provides additional generic classes such as multisets, multimaps, bitmaps, and immutable collections.

The library's design and code were advised and reviewed by Joshua Bloch, the original lead designer of the Java Collections framework, and Doug Lea, one of the lead designers of concurrency utilities in JDK.

As of April 2012, Guava ranked the 12th most popular Java library, next to the Apache Commons projects and a few others. Research performed in 2013 on 10,000 GitHub projects found that Google-made libraries, such as Google Web Toolkit and Guava, constituted 7 of the top 100 most popular libraries in Java, and that Guava was the 8th most popular Java library. As of March 2018, Guava was the 6th most starred Java project on Github.